VOID MAPPER

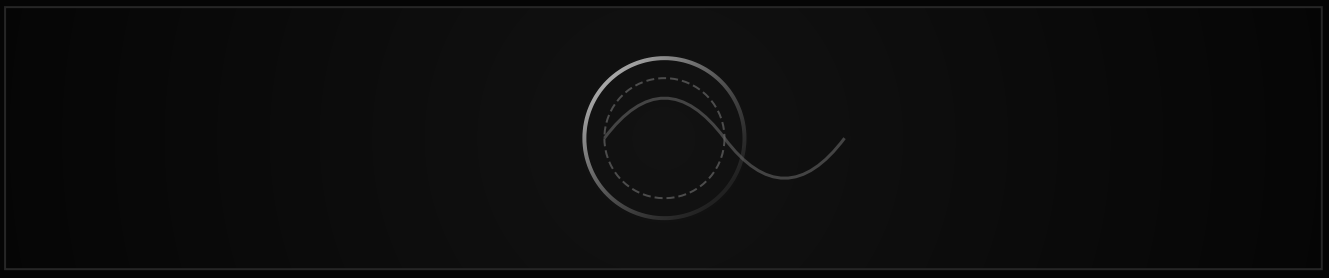
PROJECT CHARTER & TECHNICAL BLUEPRINT // INITIATIVE: VIBECODER

I. Executive Summary

The "Void Mapper" initiative is a departure from traditional, utilitarian software development. It exists entirely in the realm of aesthetic engineering and digital art. The application ingests a standard GitHub repository URL, extracts its metadata via the REST API, and procedurally generates a unique, interactive 3D WebGL artifact (an "Aura"). It translates cold, analytical data—commit histories, language usage, and issue tracking—into a living, breathing visual representation.

II. Core Objectives

- **Data Translation:** Establish a robust pipeline that maps raw JSON telemetry from GitHub into standardized vertex, material, and physics data.
- **Procedural Geometry:** Utilize React Three Fiber to build dynamic meshes that alter their physical structure based on repository age and complexity.
- **Sensory Feedback:** Ensure the final output feels ethereal and organic. A codebase with heavy technical debt should visually reflect chaos, while a highly maintained library should render as a smooth, stable monolith.



> **Aesthetic Protocol (The "Vibe")**
> The user interface must remain entirely out of the way. The background is true dark mode (pure black). There are no navbars, no extensive footers, and no intrusive analytics trackers. The focus is 100% on the rotating 3D WebGL element floating in the center of the viewport. It operates as a minimalist digital museum for code.

III. Data-to-Aura Mapping Engine

The core of this application is the translation engine. Meaningless aesthetic variation is just random noise; true procedural art requires deterministic mapping. The following table dictates exactly how GitHub API payloads map to Three.js properties.

GitHub Data Metric	JSON Endpoint / Key	Three.js Property Impact
Total Commits	`/commits` (Length/Count)	Dictates vertex density and segment geometry. A high commit count yields a smooth, high-poly sphere.
Top Languages	`/languages` (Bytes)	Determines base material grayscale tone. Maps standard language dominance to stark whites, grays, or deep blacks.
Open Issues	`open_issues_count`	Introduces noise/chaos. High open issues cause vertices to randomly offset, creating a spiked or glitched effect.
Repository Age	`created_at` vs Now	Modulates rotational velocity and ambient pulse rate. Older repos rotate slowly; new ones spin rapidly.

IV. Technical Specifications

This project intentionally isolates backend complexity by relying entirely on client-side fetching and rendering. The architecture is a pure single-page application (SPA) acting as a visualizer, with no proprietary database required to store state.

The Harvester (API Layer)
The application will utilize the standard public GitHub REST API v3. To avoid immediate rate-limiting (60 requests per hour for unauthenticated clients), the fetching logic must be debounced and cache results in the browser's `sessionStorage`. If a repository has already been parsed during the session, the application will retrieve the cached aura state instead of initiating a new network request.

WebGL & Shader Complexities
While basic geometries (boxes, spheres) are trivial in React Three Fiber, achieving the stark, luminous glow requires post-processing. The application must implement the `@react-three/postprocessing` library, specifically utilizing the `Bloom` and `Noise` effects pass. These post-processing effects are critical for elevating the artifact from a basic 3D model to a vibrant piece of digital art.

V. Core Technology Stack

Vite + React
The foundation of the SPA. Vite provides hot-module replacement for rapid iteration, which is essential when tweaking 3D lighting and material colors in real-time.

React Three Fiber (R3F)
A React renderer for Three.js. It allows us to build complex 3D scenes declaratively, binding state directly to mesh properties without manually writing recursive render loops.

Tailwind CSS
Used strictly for the minimalist UI overlay. Absolute positioning, z-indexing, and clean dark-mode input fields will be handled via utility classes to keep CSS bundle size negligible.

GitHub REST API
The sole data provider. The fetch layer will asynchronously gather commits, languages, and repo metadata to populate the master state object prior to canvas initialization.

VI. Development & Implementation Phases

Phase 1: Environment & Shell `DAYS 1-2`
Initialize Vite project. Build the stark, pure black user interface. Establish the input field capable of parsing standard GitHub URLs (e.g., `Pranshu125/J.A.R.V.I.S`) into owner and repo variables.

Phase 2: Data Ingestion `DAYS 3-4`
Write the asynchronous `fetchRepositoryData` function. Consolidate the JSON responses from three separate GitHub endpoints into a single normalized `AuraState` object ready for the renderer.

Phase 3: The WebGL Forge `DAYS 5-7`
Mount the R3F <Canvas>. Inject the `AuraState` to manipulate a `MeshStandardMaterial`. Apply directional lighting, implement the rotation via `useFrame`, and attach post-processing bloom.

VII. AI Execution Protocol

To accelerate prototyping, this exact prompt should be fed into a primary generative development tool (like Google Antigravity, Claude, or GitHub Copilot Workspace) to bootstrap the core scaffolding in a single pass.

Prompt to Antigravity Generator:

```
Act as a senior frontend architect and WebGL specialist. I am building the 'Void Mapper WebGL Generator' as a Vite + React application. Task 1: Generate the initialization configuration for a Vite React project including necessary dependencies: three, @react-three/fiber, @react-three/drei, @react-three/postprocessing, and tailwindcss. Task 2: Write the complete `App.jsx`. It must include a minimal dark-mode (pure black and white) UI with a single text input for a GitHub 'owner/repo' string. Task 3: Implement an asynchronous fetch layer to hit the GitHub API `/repos/{owner}/{repo}` and `/languages` endpoints. Task 4: Build the `AuraCanvas` component using React Three Fiber. Map the fetched GitHub repository size to a slowly rotating, floating `IcosahedronGeometry`. If the repo has high open issues, add a distortion effect to the mesh. Apply a heavy Bloom effect via post-processing to give it an ethereal white glow. Ensure the canvas fills the background.
```

VIII. Deployment & Version Control

Upon generating the codebase locally, execute the following commands to initialize the repository, stage the aesthetic prototype, and push it directly to the designated GitHub endpoint.

```
cd void-mapper git init git add . git commit -m "Initial commit: Core WebGL logic and GitHub API ingestion for Void Mapper" git branch -M main git remote add origin https://github.com/Pranshu125/void-mapper.git git push -u origin main
```

Next Steps Post-Deployment

Once the initial commit is pushed, verify that the canvas mounts locally via `npm run dev`. The immediate next phase will involve refining the mapping algorithms to ensure that extreme data outliers (e.g., repositories with 10,000+ commits) do not crash the WebGL context by generating overly dense polygons.